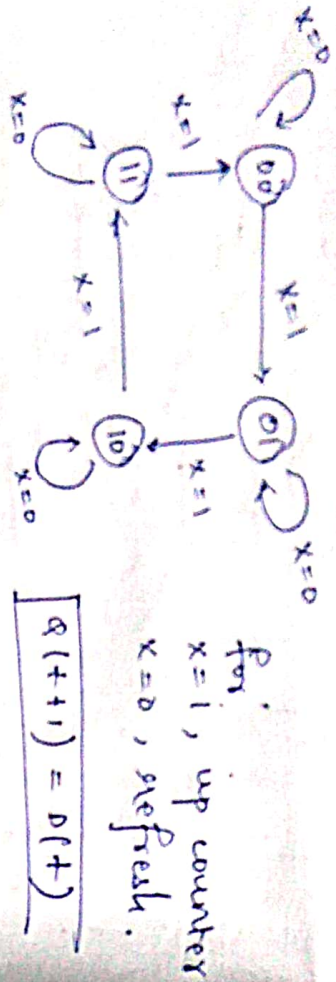


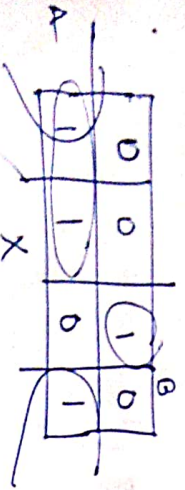
* Implementation of the state diagram using DFF (D-Flip-Flop).



Present state	input	Next state
A	X	A(t+1) B(t+1)
0	0	0 0
0	1	0 1
1	0	1 0
1	1	1 1

Present state	input	Next state
A	X	A(t+1) B(t+1)
0	0	0 0
0	1	0 1
1	0	1 0
1	1	1 1

$$D_A = A\bar{B} + \bar{A}X + \bar{A}BX$$



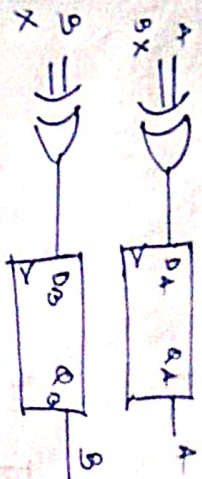
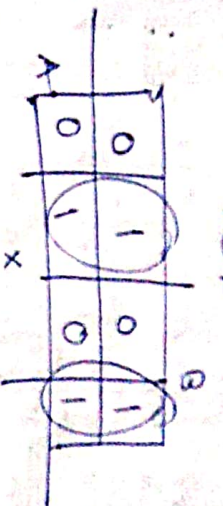
$$D_A = A(\bar{B} + X) + \bar{A}BX$$

$$= A\bar{B}X + \bar{A}BX$$

$$= A \oplus BX$$

$$D_B = \bar{A}X + B\bar{X}$$

$$= B \oplus X$$



* Implement the state diagram using TFF

$$Q_A(t+1) = T \oplus Q$$

$$Q_A = Q \text{ for } T=0$$

$$Q_A = \bar{Q} \text{ for } T=1$$

Don't Care.

Present state	input	Next state
A	B	A(t+1) B(t+1)
0	0	0 0
0	1	0 1
1	0	1 0
1	1	1 1

refresh 0 0 1 1

refresh 0 1 0 1

refresh 1 0 1 0

refresh 1 1 0 1

same 0 0 1 1

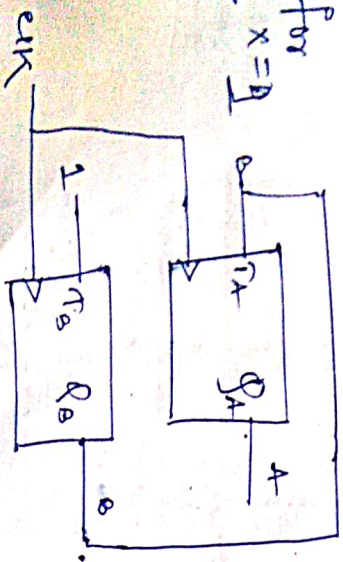
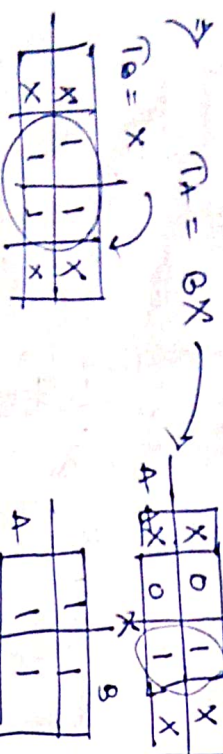
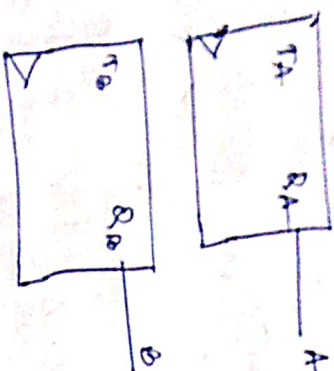
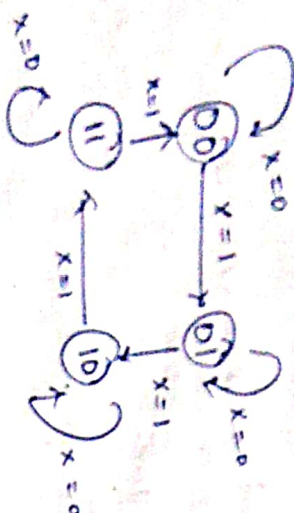
same 1 0 0 1

same 1 1 0 0

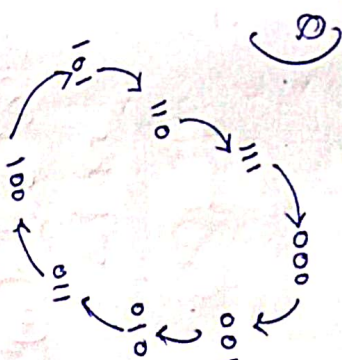
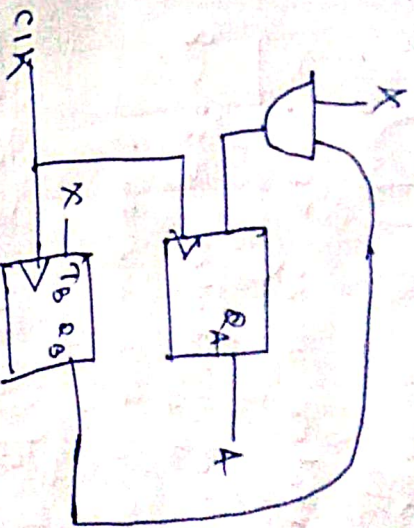
same 1 1 1 1

if flipping in next state then 1 if not 0

It is up counter



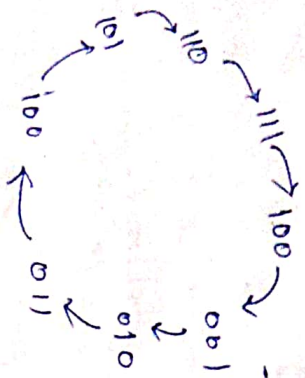
see next page



draw it.

A B C	T_A	T_B	T_C	Q_A	Q_B	Q_C
from dia.	?	?	?			from dia.

* 3-bit up counter using TFF :-
 no. of states = $2^3 = 8$



$$A_i(t+1) = T A_i(t) A_i$$

state diagram

present state	next state	inputs
$A_3 A_2 A_1$	$A_3(t+1) A_2(t+1) A_1(t+1)$	$T A_3 T A_2 T A_1$
000	001	0 0 0
001	010	0 0 1
010	011	0 1 0
011	100	0 1 1
100	101	1 0 0
101	110	1 0 1
110	111	1 1 0
111	000	1 1 1

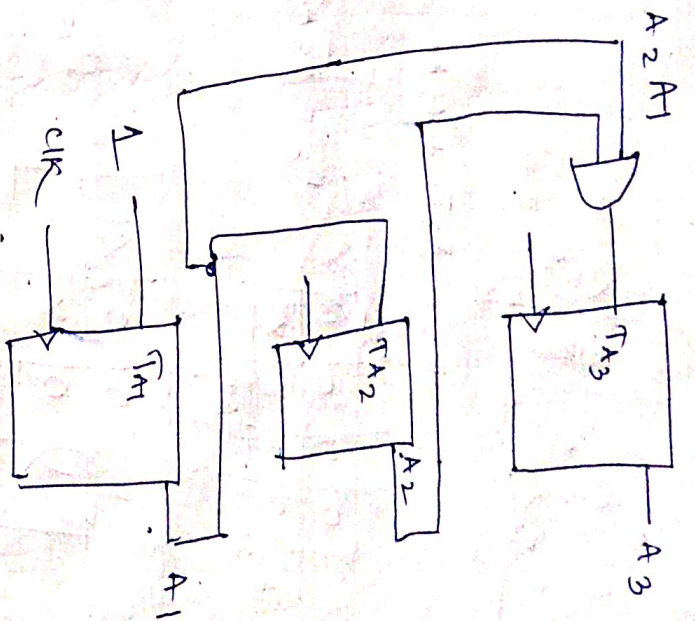
A_3	A_2	A_1
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$$T A_3 = A_2 A_1$$

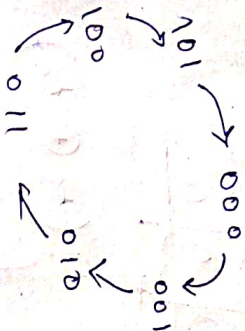
A_3	A_2	A_1
0	0	0
0	1	1
1	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$$T A_2 = A_1$$

$$T A_1 = 1$$



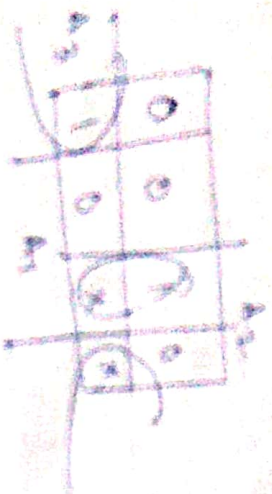
* Count 0 to 5 using DFF :-



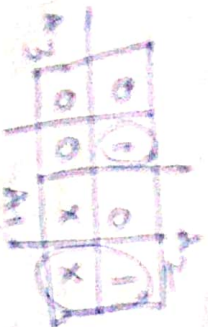
$$A_i(t+1) = D_i$$

present state	next state
$A_3 A_2 A_1$	$A_3 A_2 A_1$
000	001
001	010
010	100
100	101
101	110
110	000

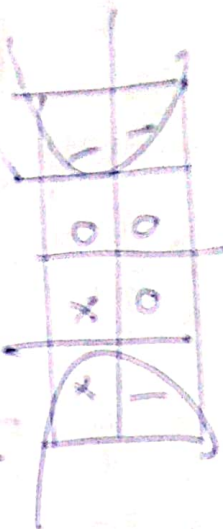
A_3	A_2	A_1
0	0	0
0	0	1
0	1	0
1	0	0
1	0	1
1	1	0
1	1	1



$$D_3 = A_3 \bar{A}_1 + A_2 A_1$$

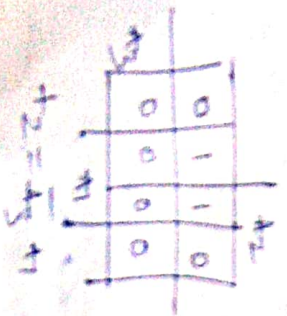


$$D_2 = A_3 A_2 \bar{A}_1 + A_2 \bar{A}_1$$

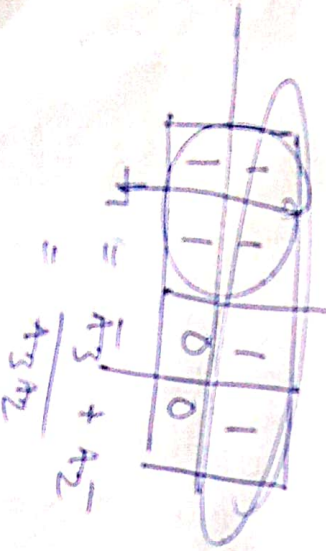
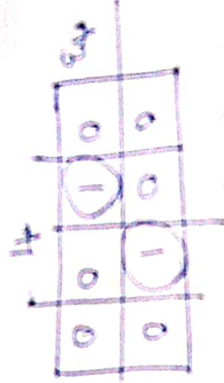


$$D_1 = \bar{A}_1$$

using KFF



$$F_3 = A_1 (A_2 \oplus A_2)$$

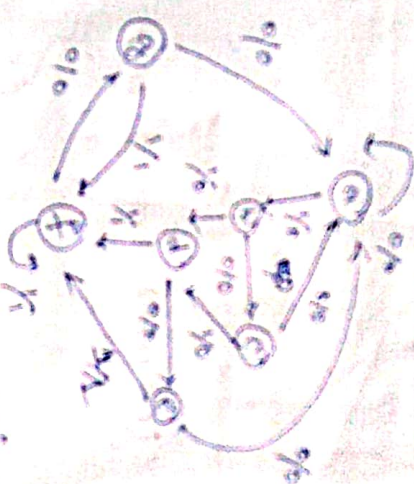


$$F_2 = \bar{A}_1 + A_2$$

state reduction

if nearly machine (input & state dependent)

more machine (depends on state).



state diagram using state table.

present state

next state

output

write in seq.

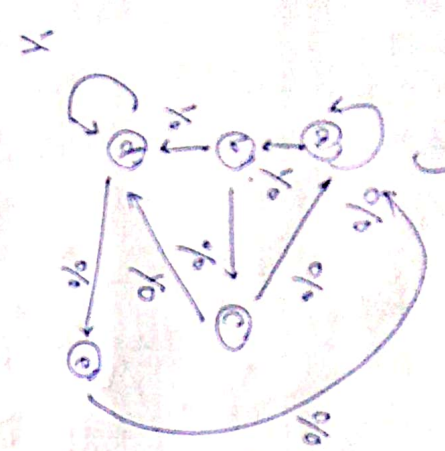
redundant

$e = 8$

present state	next state	output
a	a	b
b	c	d
c	d	e
d	e	f
e	f	a
f	a	b

replace all the g with f.

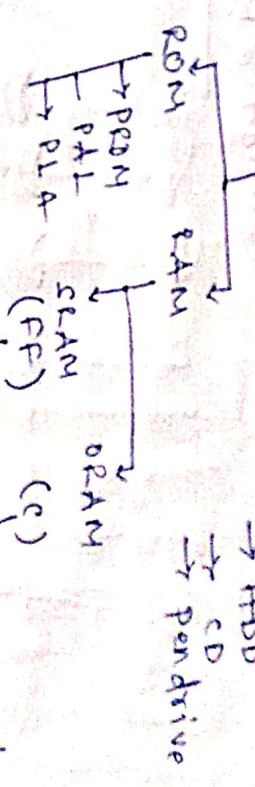
replace f with d.



g & f are redundant state.

Memory

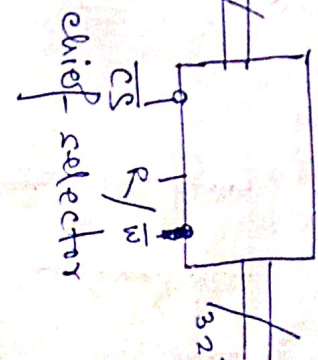
primary secondary



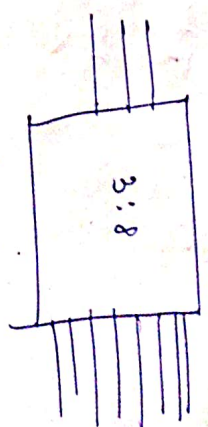
Read only memory

EPROM
EEPROM
EEPROM

address bus

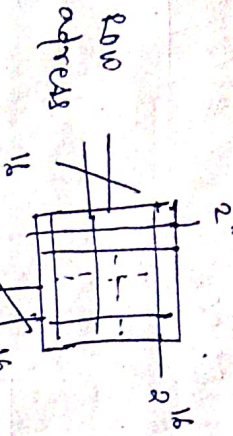


data bus



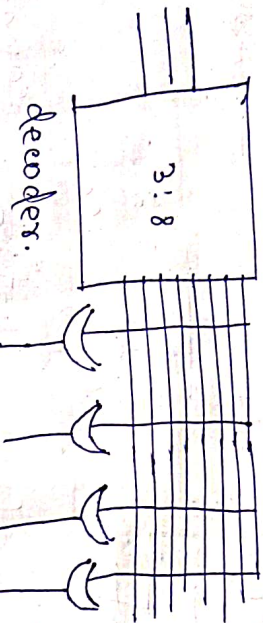
decoder 8 byte

$2^{32} \times 32$ bits
 $4k \times 32$ bits

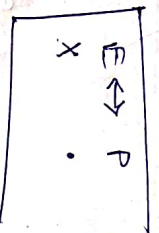
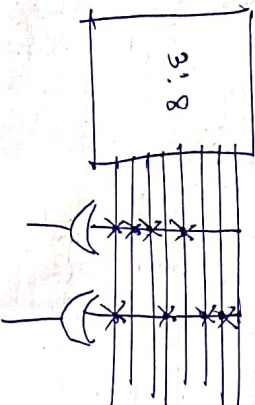


each intersection provides a ROM.

programming of ROM

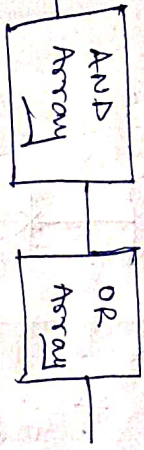
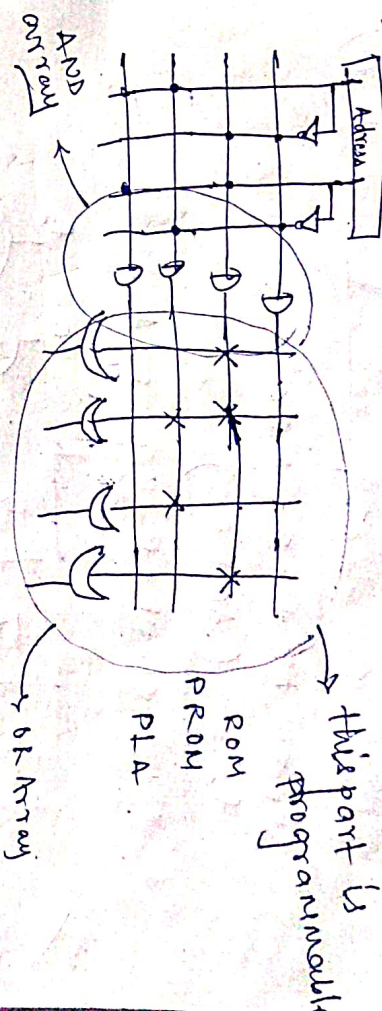


Full adder (carry, sum)
Full subtractor (diff, borrow).



* PAL: 'programmable array logic'
PLA: 'programmable logic array'

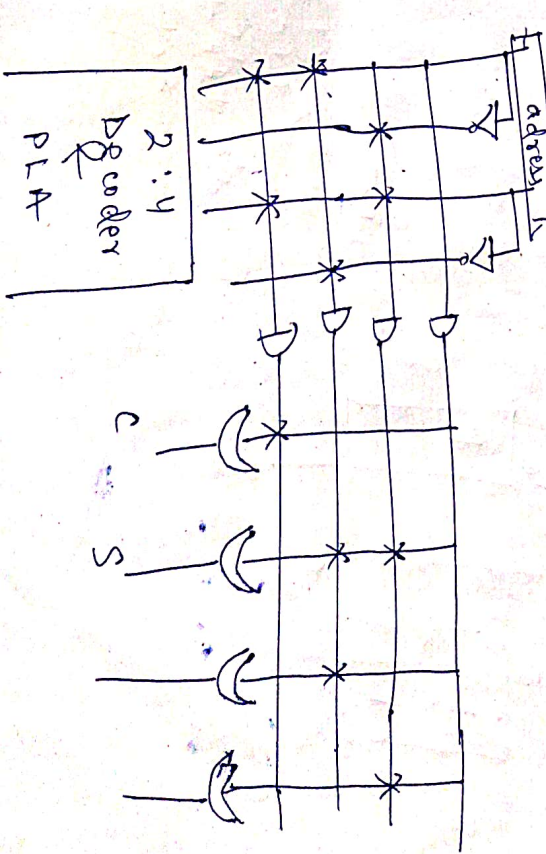
* 2:4 decoder



AND Array
OR Array
Fixed
programmable
Fixed
programmable
ROM
PAL
PLA
How to remember
RD -> Read
A -> AND
program-program

configuring PLA to half adder.

PLA -> half adder.

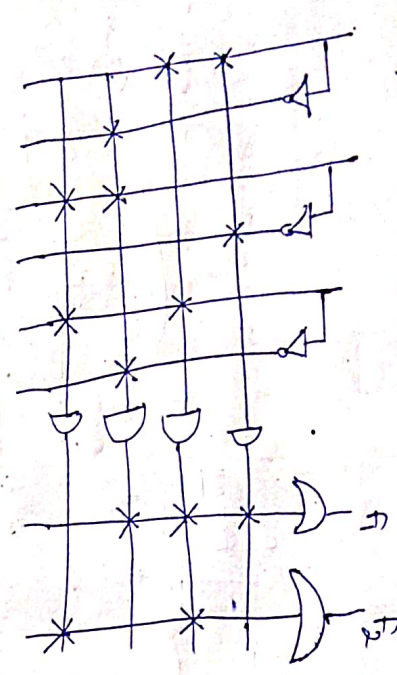


21)

$$F_1 = AB' + AC + A'Bc'$$

$$F_2 = (AC + BC)$$

Implement using PLA

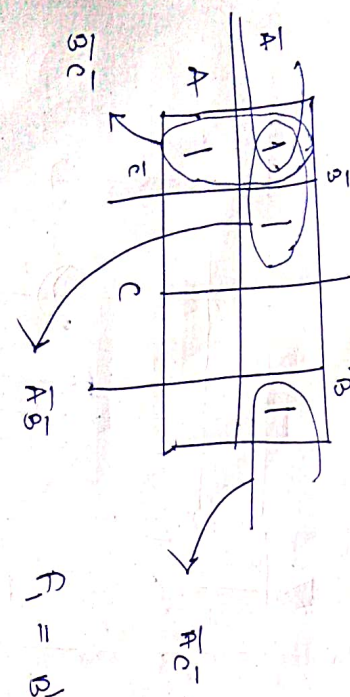


22)

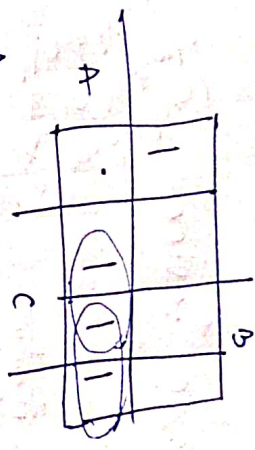
$$F_1(A, B, C) = \sum_m (0, 1, 2, 4)$$

$$F_2(A, B, C) = \sum_m (0, 5, 6, 7)$$

Implement using PLA



$$F_1 = Bc' + A'B' + A'C'$$



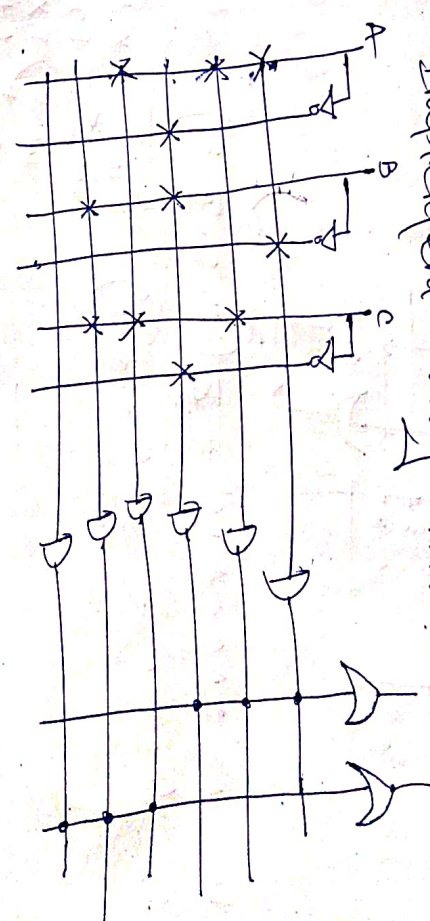
$$F_2 = A'Bc' + AB + AC$$

23)

$$F_1 = AB' + AC + A'Bc'$$

$$F_2 = AC + BC$$

Implement using PAL



Drawbacks of PAL:

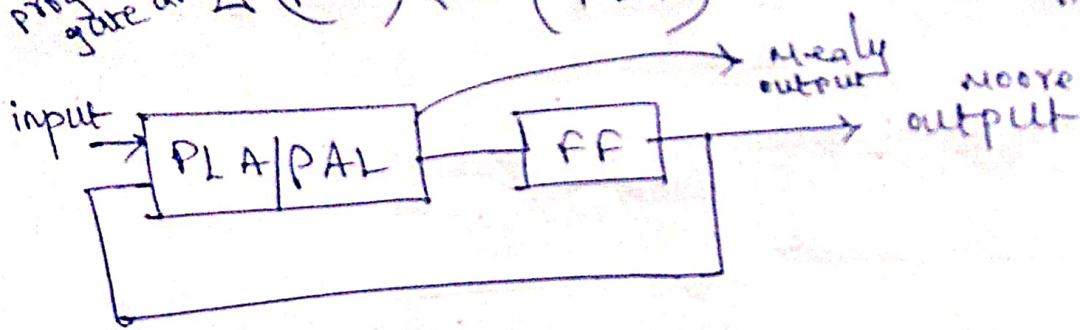
- 1) reuse
- 2) unused?

field programmable gate array (FPGA)

Programmable logic device

sequential logic / state machine

(PLD)

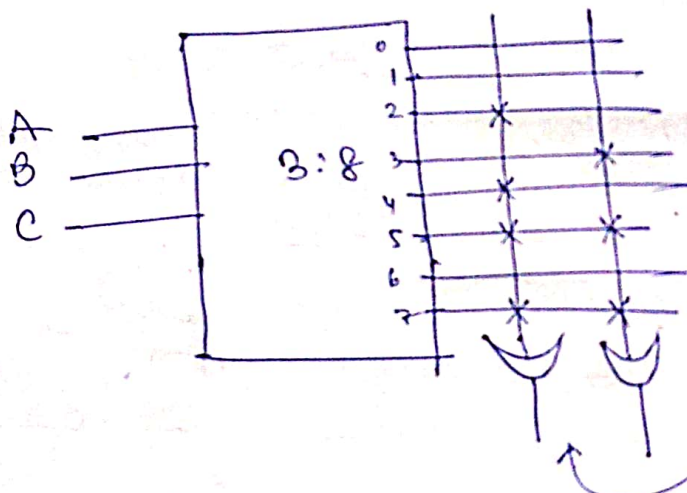


Q4)

$$F_1 = AB' + AC + A'BC'$$

$$F_2 = (AC + BC)$$

Implement using PROM.



$$F_1 = AB' + AC + A'BC' = \sum_m (4, 5, 7, 2)$$

$$F_2 = (AC + BC) = \sum_m (5, 7, 3)$$

100	X 11
101	10 X
101	1 X 1
111	0 1 0

class test :

Tuesday

sequential,

memory, counter, latch, FF,

State machine, register